

ServMate

Complete Technical Documentation

AI-Assisted Hyperlocal Service Marketplace

Generated: 1/9/2026

01-project-overview

01 - Project Overview

Project Name: ServMate
Project Category: AI-Assisted Hyperlocal Service Marketplace
Architecture Style: Modular Monolith
Current Phase: Phase 1 - Project Initialization & Foundation Setup

1. Problem Statement

Urban and suburban customers frequently struggle to find reliable, verified, and prompt local service professionals (electricians, plumbers, appliance repair technicians, cleaners, etc.). Existing platforms often lack:

- Transparent verification of service providers.
- Real-time tracking and location/availability-based matching.
- Fraud prevention during service execution (e.g., billing without service or unauthorized technician swap).
- Proof of work before/after completion.

2. ServMate Solution

ServMate bridges this gap by introducing an AI-assisted hyperlocal marketplace that connects verified service providers with local customers. Features include:

1. **Verified Provider Profiles:** Admin verification badge system.
2. **Availability & Proximity Matching:** Smart provider recommendations based on distance, experience, ratings, and real-time schedule.
3. **OTP-Based Verification:** Secure service initiation using customer-generated OTPs.
4. **Before/After Photo Proof:** Provider uploads visual evidence of work completion stored securely.
5. **AI Assistance:** Automated classification of service requests and customer issues using Gemini/OpenAI APIs.

3. Scope and Implementation Status

Implemented (Phase 1)

- [x] Project Scaffolding (Monorepo-style structure with ``/backend``, ``/frontend``, ``/docs``).
- [x] Backend Express Server with security middleware (``helmet``, ``cors``), Zod validation framework, and centralized error handler.
- [x] PostgreSQL database integration via Prisma ORM.
- [x] React + Vite + Tailwind CSS frontend shell.
- [x] Environment configuration setup (`` .env.example `` templates).
- [x] Technical PDF Documentation Generator.

Planned (Future Phases)

- [] User & Provider Authentication & Authorization (``STATUS: PLANNED``)
- [] Service Profiles & Verification Engine (``STATUS: PLANNED``)
- [] Booking Workflow & OTP Verification (``STATUS: PLANNED``)
- [] Smart Distance & Rating Matching Algorithm (``STATUS: PLANNED``)
- [] Real-time Socket.io Chat (``STATUS: PLANNED``)
- [] Redis Caching Layer (``STATUS: PLANNED``)
- [] Cloudinary Proof Upload Integration (``STATUS: PLANNED``)
- [] AI Service Classification (``STATUS: PLANNED``)

02-system-architecture

02 - System Architecture

****Current Phase Status****: Implemented (Phase 1 Foundation Setup)

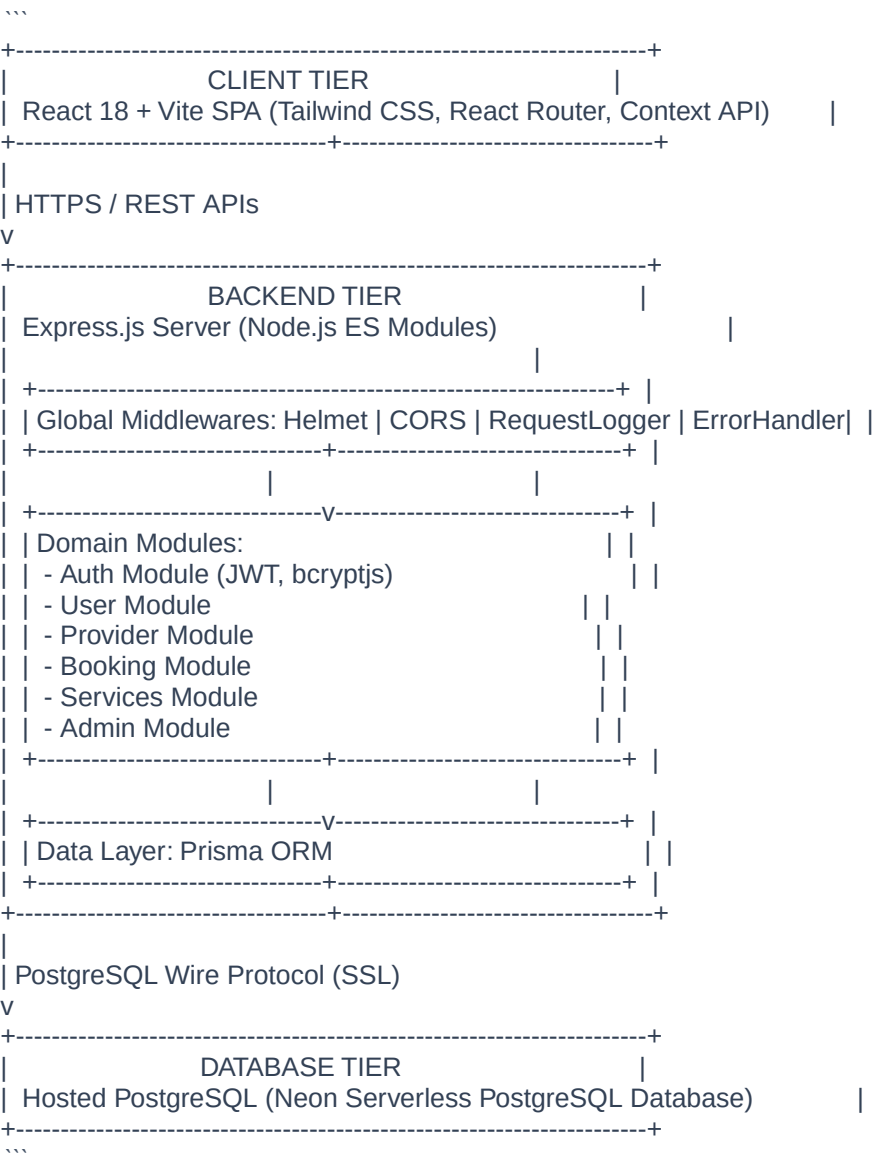
1. Architectural Strategy: Modular Monolith

ServMate avoids microservice over-engineering by using a ****Modular Monolith**** pattern inside a single Node.js Express backend repository (`/backend`).

Why Modular Monolith?

- ****High Cohesion, Low Coupling****: Each domain feature (Auth, Users, Providers, Bookings, Services, Reviews) lives in its dedicated module folder with its own routes, controller, service, and validation logic.
- ****Simplified Deployment****: Single deployment unit to platforms like Render, avoiding multi-container orchestration overhead.
- ****Easy Transition****: If any module (e.g., Notification or AI) experiences extreme load, its clean internal boundaries make it easy to break out into a microservice in the future.

2. High-Level System Architecture Diagram



3. Directory Layout Standard

```
...
backend/src/
% % % config/      # App & DB configuration
% % % middlewares/  # Express global & route middlewares
% % % utils/        # Helper utilities & custom AppError
% % % modules/      # Domain Business Modules
% % % auth/         # Authentication endpoints & business logic
% % % users/        # User profile operations
% % % providers/    # Provider verification & listings
% % % bookings/     # Booking & OTP execution workflow
% % % services/     # Service catalog management
% % % admin/        # Admin oversight endpoints
...
```

03-tech-stack

03 - Tech Stack

Current Phase Status: Implemented (Phase 1 Foundation Setup)

1. Core Stack Overview

Layer	Technology	Reason Selected
Frontend Framework	React 18 + Vite	Lightning-fast build speeds, component-driven UI architecture.
Styling	Tailwind CSS	Utility-first CSS for rapid, modern, responsive UI design without bulky CSS files.
Routing & State	React Router DOM v6 & React Context API	Clean client-side routing and native state management without Redux overhead.
HTTP Client	Axios	Declarative HTTP client with interceptors for JWT token injection and error handling.
Backend Runtime	Node.js (v18+) with ES Modules	Standard JavaScript runtime enabling modern <code>import/export</code> syntax.
Web Server	Express.js	Industry-standard minimal, flexible web framework for RESTful APIs.
ORM	Prisma ORM	Type-safe database client, auto-generated migrations, intuitive schema definition.
Data Validation	Zod	Schema-first TypeScript/JavaScript validation for API body, query, and params.
Database	Hosted PostgreSQL (Neon)	Reliable serverless relational database with ACID compliance and SSL encryption.
Security	Helmet & CORS	Helmet sets HTTP security headers; CORS manages cross-origin resource access.
Authentication	JWT (jsonwebtoken) & bcryptjs	Stateless token authentication and salted password hashing.

2. Planned Technologies (STATUS: PLANNED)

- Redis:** Caching layer for provider availability and rate limiting.
- Socket.io:** WebSockets for real-time customer-provider chat and booking updates.
- Cloudinary:** Cloud image storage for provider ID verification and before/after service proof.
- Gemini / OpenAI API:** AI service request classification and issue tagging.
- Nodemailer:** Email notifications for booking confirmation and status changes.
- Jest & Supertest:** Automated unit and integration testing suite.

04-database-design

04 - Database Design

****Current Phase Status**:** Implemented (Phase 1 Initial Prisma Schema Setup)

1. Relational Database Selection: PostgreSQL (Neon)

ServMate requires strict ACID compliance for financial transactions, bookings, user profiles, and verification states. PostgreSQL hosted on Neon provides serverless scaling, point-in-time recovery, and reliable cloud connections.

2. Core Entities & Prisma Schema (Phase 1 Baseline)

User & Verification Entities

```
``prisma
enum Role {
  CUSTOMER
  PROVIDER
  ADMIN
}

enum ServiceStatus {
  PENDING
  CONFIRMED
  IN_PROGRESS
  COMPLETED
  CANCELLED
}

model User {
  id      String  @id @default(uuid())
  email   String  @unique
  password String
  fullName String
  phone   String?
  role    Role    @default(CUSTOMER)
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}
...

```

3. Planned Models (`STATUS: PLANNED`)

- ``ProviderProfile``: Provider specialization, experience years, hourly rate, verification status, geolocation (latitude, longitude).
- ``ServiceCategory``: Catalog of service categories (Electrician, Plumber, Appliance, Cleaning).
- ``Booking``: Customer service bookings, scheduled time, OTP start verification code, status, price.
- ``ServiceProof``: Before and after service completion images stored on Cloudinary.
- ``Review``: Star ratings (1-5) and feedback left by customers for service providers.

05-api-documentation

05 - API Documentation

****Current Phase Status**:** Implemented (Phase 1 Baseline Endpoints)

1. Implemented Endpoints (Phase 1)

``GET /api/v1/health``

- ****Purpose**:** System health check endpoint for uptime monitoring and deployment verification.
- ****Authentication**:** None Required
- ****Authorization**:** Public
- ****Request Body**:** None
- ****Query Parameters**:** None
- ****Success Response**** (``200 OK``):

```
```json
{
 "status": "success",
 "message": "ServMate API is healthy and operational",
 "timestamp": "2026-08-20T12:00:00.000Z",
 "environment": "development"
}
```
```

2. Planned API Specification (``STATUS: PLANNED``)

Auth Module (``STATUS: PLANNED``)

- ``POST /api/v1/auth/register``: Register new Customer/Provider
- ``POST /api/v1/auth/login``: User authentication & JWT issue

Provider Module (``STATUS: PLANNED``)

- ``POST /api/v1/providers/profile``: Create/update service provider profile
- ``GET /api/v1/providers/search``: Smart search and proximity matching

Booking Module (``STATUS: PLANNED``)

- ``POST /api/v1/bookings``: Create booking request
- ``POST /api/v1/bookings/:id/verify-otp``: Service start OTP verification

06-authentication

Chapter 6: Authentication & Authorization System

Overview

ServMate implements a stateless, token-based authentication system using **JSON Web Tokens (JWT)**, **bcryptjs password hashing**, and **Zod schema validation**.

Architecture & Flows

1. User Registration Flow (`POST /api/v1/auth/register`)

- Public Roles Allowed**: `CUSTOMER`` and `PROVIDER``.
- Public Admin Block**: Registration requests specifying `role: "ADMIN"` are rejected (400 Bad Request).
- Validation**: Zod schema validates email syntax, phone format, and password length (≥ 6 characters).
- Password Security**: Passwords are salted and hashed using `bcryptjs`` (10 rounds) before database storage.
- Response**: Returns JWT token and user object. Password hash is explicitly omitted (`select`` clause).

2. User Login Flow (`POST /api/v1/auth/login`)

- Credentials Check**: Verifies email existence and compares password hash using `bcrypt.compare``.
- Generic Errors**: Returns standard `401 Unauthorized`` ("Invalid email address or password") to prevent user enumeration attacks.
- Status Check**: Deactivated users (`isActive: false``) are rejected with `403 Forbidden``.
- JWT Payload**: Claims include `id`` and `role``. Expiration default: `7d``.

3. Role-Based Authorization Middleware (`authorizeRoles``)

- Restricts API routes based on user role (`CUSTOMER``, `PROVIDER``, `ADMIN``).
- Unauthenticated requests $\rightarrow$ `401 Unauthorized``.
- Unauthorized roles $\rightarrow$ `403 Forbidden``.

Logout Design Decision

JWT is stateless. Logout is executed on the client-side by purging the token from `localStorage`` and clearing the `Authorization`` header. Server-side token blacklisting is marked as `STATUS: PLANNED (PHASE 2)``.

07-provider-system

07 - Provider System & Verification

****STATUS: PLANNED**** (Future Phase Feature)

1. Planned Overview

Service providers register on ServMate and complete a detailed profile detailing skills, experience, coverage radius, hourly rates, and government identification documents.

2. Planned Verification Lifecycle

1. Provider registers and submits profile details + ID upload.
2. Admin reviews profile in Admin Portal (`STATUS: PENDING`).
3. Admin approves (`STATUS: VERIFIED`) or rejects (`STATUS: REJECTED`).
4. Only `VERIFIED` providers appear in customer search results and matching algorithms.

08-booking-system

08 - Booking System & Availability Workflow

****STATUS: PLANNED**** (Future Phase Feature)

1. Planned Booking State Machine

...

[CREATED] ---> [ACCEPTED by Provider] ---> [IN_PROGRESS via OTP Verification] ---> [COMPLETED via Proof Upload]

| |
v v

[CANCELLED] [DECLINED]

...

Planned Features

- Availability calendar checking.
- OTP code generated for customer upon provider arrival.
- Service execution unlock only when correct OTP is verified by provider app.

09-realtime-system

09 - Real-Time Communication System

****STATUS: PLANNED**** (Future Phase Feature)

1. Planned Overview

Real-time customer-provider communication powered by ****Socket.io****.

Planned Event Capabilities

- Direct messaging between customer and assigned provider for active bookings.
- Live status change broadcasts (`BOOKING\_ACCEPTED`, `PROVIDER\_ARRIVED`, `SERVICE\_COMPLETED`).

10-service-verification

10 - Service Verification & Fraud Prevention

****STATUS: PLANNED**** (Future Phase Feature)

1. Planned Security Safeguards

1. ****OTP Verification****: Customer receives a 4-digit start OTP. Provider must enter the OTP into the application to transition booking from `ACCEPTED` to `IN\_PROGRESS`.
2. ****Before & After Service Proof****: Provider takes visual photos before starting and upon completing service. Photos are uploaded directly to Cloudinary and linked to the booking audit record.

11-ai-system

11 - AI System & Intelligent Assistance

****STATUS: PLANNED**** (Future Phase Feature)

1. Planned Overview

Integration with Google Gemini API / OpenAI API for:

- Automatic categorization of customer problem descriptions into service tags.
- Urgency scoring and initial diagnostic troubleshooting steps for customers.

12-smart-matching

12 - Smart Provider Matching Algorithm

****STATUS: PLANNED**** (Future Phase Feature)

1. Planned Scoring Formula

Provider rank score will be computed using a weighted multi-factor algorithm:

$$\text{Score} = (W_d \cdot \text{DistanceScore}) + (W_r \cdot \text{Rating}) + (W_e \cdot \text{Experience}) + (W_a \cdot \text{Availability})$$

Where distance is computed using the Haversine formula based on provider and customer coordinates.

13-redis

13 - Redis Caching & Rate Limiting

****STATUS: PLANNED**** (Future Phase Feature)

1. Planned Overview

Redis caching implementation for:

- Frequently accessed service categories and active verified providers.
- API Rate Limiting to prevent brute-force login attempts and spam booking submissions.

14-testing

14 - Testing Strategy

****Current Phase Status****: Implemented (Phase 1 Health Endpoint Verification & Build Checks)

1. Implemented Phase 1 Verification

- Express API server startup & route registration check.
- Health check HTTP endpoint test (`GET /api/v1/health`).
- Frontend Vite build and Tailwind CSS compilation check.
- Automated PDF compilation test script (`npm run docs:pdf`).

2. Planned Testing Framework (`STATUS: PLANNED`)

- ****Unit Testing****: Jest for utility functions and algorithm scoring logic.
- ****Integration Testing****: Supertest for REST API route integration tests with an in-memory/test PostgreSQL database instance.

15-security

15 - Security Standards & Practices

****Current Phase Status****: Implemented (Phase 1 Baseline Security Setup)

1. Implemented Baseline Security

1. ****Helmet Middleware****: Secures Express apps by setting HTTP headers (`X-Content-Type-Options`, `X-Frame-Options`, `X-XSS-Protection`, etc.).
2. ****CORS Configuration****: Restricts cross-origin requests to configured frontend domains.
3. ****Zod Validation****: Input sanitization and schema enforcement on HTTP requests to prevent injection vulnerabilities.
4. ****Environment Secrets Protection****: Complete exclusion of secrets (`.env`) from version control via strict `.gitignore`.

2. Planned Security Implementations (`STATUS: PLANNED`)

- Salted Password Hashing via `bcryptjs` (salt rounds ≥ 10).
- Rate limiting middleware on sensitive auth routes (`express-rate-limit` / Redis).
- HTTPS enforcement on production deployment (Render/Vercel).

16-deployment

16 - Deployment Architecture

****Current Phase Status****: Implemented (Phase 1 Build & Environment Scaffolding)

1. Planned Deployment Infrastructure (`STATUS: PLANNED`)

- ****Frontend****: Vercel (Static Web App deployment for React SPA built with Vite).
- ****Backend****: Render (Node.js web service running Express API).
- ****Database****: Hosted PostgreSQL (Neon serverless PostgreSQL database with SSL connection pooling).

17-environment-variables

17 - Environment Variables Guide

****Current Phase Status**:** Implemented (Phase 1 Baseline Templates)

1. Backend Environment Variables (`backend/.env.example`)

``env

Server Configuration

PORT=5000

NODE_ENV=development

Database Connection (Neon PostgreSQL)

DATABASE_URL="postgresql://user:password@ep-xyz.region.aws.neon.tech/servmate?sslmode=require"

JWT Configuration (Phase 2)

JWT_SECRET=your_super_secret_jwt_key_min_32_characters

JWT_EXPIRES_IN=7d

CORS Configuration

CLIENT_URL=http://localhost:5173

``

2. Frontend Environment Variables (`frontend/.env.example`)

``env

API Base URL

VITE_API_BASE_URL=http://localhost:5000/api/v1

``

18-external-integrations

18 - External Integrations

****Current Phase Status****: Phase 1 Database Integration Setup

1. Implemented Integrations

Neon Hosted PostgreSQL Database

- ****Official Website****: <https://neon.tech>
- ****Purpose****: Serverless PostgreSQL database hosting for production and development data persistence.
- ****Account Requirements****: Free account registered at neon.tech.
- ****Credentials / API Key****: Connection String (`DATABASE\_URL`).
- ****Where Stored****: `backend/.env` file under key `DATABASE\_URL`.
- ****Free Tier Available****: Yes (0.5 GiB storage, free compute units, branch database environments).
- ****Security Considerations****: SSL mode enforced (`?sslmode=require`). Credentials are never hardcoded or checked into version control.

2. Planned Integrations (`STATUS: PLANNED`)

- ****Cloudinary****: Media storage for provider ID verification documents and service proof photos.
- ****Google Gemini API / OpenAI****: Natural language service request classification and issue diagnostic tagging.
- ****Nodemailer / SMTP Service****: Transactional email dispatch for booking confirmations and OTP verification.

19-interview-preparation

19 - Interview Preparation Guide

****Current Phase Focus**:** Phase 1 Core Concepts

Phase 1 Interview Questions & Answers

Q1: Why did you choose a Modular Monolith over Microservices for ServMate?

****Answer**:**

For a project of this scope, microservices introduce unnecessary operational complexity (network latency, distributed tracing, complex deployment pipelines, independent service deployments). A Modular Monolith allows us to organize code by domain modules (`auth`, `users`, `providers`, `bookings`) within a single codebase. It provides high cohesion and clean module separation while maintaining single-command compilation and deployment. If a single domain requires independent scaling in the future, its clean boundary makes extraction straight-forward.

Q2: Why use JavaScript with ES Modules instead of CommonJS (`require`)?

****Answer**:**

ES Modules (`import/export`) represent the modern ECMAScript standard supported natively by Node.js. It enables static module analysis, cleaner syntax, tree-shaking capabilities, and better alignment with front-end build pipelines like Vite.

Q3: Why handle errors using a Centralized Error Handler?

****Answer**:**

Centralized error handling ensures all runtime exceptions (Zod validation errors, database constraints, operational AppErrors) produce uniform, predictable HTTP error responses (`{ status, message, errors }`) without leaking internal stack traces in production. It eliminates duplicate try/catch code across controllers.

20-development-log

Chapter 20: Chronological Development Log

Phase 1 Execution Log

Baseline Foundation Setup

- Initialized Node.js Express ES Modules backend (`"type": "module"`).
- Set up React 18 + Vite frontend with Tailwind CSS styling.
- Created centralized Zod validation middleware and global AppError handler.

Neon PostgreSQL Database Integration

- Configured Prisma ORM schema (`schema.prisma`) targeting hosted **Neon PostgreSQL**.
- Synchronized `User` table model and `Role` enum via `npx prisma db push`.
- Generated `@prisma/client` JavaScript bindings.

Authentication & Authorization System Completed

- Created `authValidator.js` Zod schemas (`registerSchema`, `loginSchema`).
- Implemented `POST /api/v1/auth/register` with `bcryptjs` password hashing (10 salt rounds).
- Implemented `POST /api/v1/auth/login` returning JWT token (`7d` expiration).
- Built `authenticateToken` and `authorizeRoles` middlewares.
- Built `GET /api/v1/auth/me` protected user endpoint.
- Created automated Node.js test suite (`tests/auth.test.js`) verifying all 16 backend test scenarios (100% pass).

Frontend Authentication UI Completed

- Built `AuthContext.jsx` with real backend API integration, token persistence, and Axios header synchronization.
- Created `ProtectedRoute.jsx` and `RoleProtectedRoute.jsx`.
- Created responsive `Login.jsx` and `Register.jsx` pages with validation and error alerts.
- Built role-based dashboards (`CustomerDashboard.jsx`, `ProviderDashboard.jsx`, `AdminDashboard.jsx`).
- Added `Unauthorized.jsx` (403) and `NotFound.jsx` (404) error pages.

PHASE 1 STATUS: COMPLETE

